Grid layout

In the last lesson, Flexbox helped you arrange items in one direction: a row of buttons, a row of cards, or a small footer inside a card.

Grid is for layouts where rows and columns matter at the same time.

Use Grid when you want a page region, dashboard, gallery, pricing table, or card layout to have a clear two-dimensional structure. Flexbox asks, "How should this group flow along one axis?" Grid asks, "What columns and rows should this space have?"

Start with a page that needs structure

The easiest way to feel Grid is to take a page that almost works and give it structure. This dashboard has a sidebar, a workspace, stat cards, and lesson cards. Without Grid, those pieces stack; with Grid, the page gets columns, rows, and deliberate gaps.

Use this as `index.html`:

html

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Grid practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Organize a learning dashboard</h1>
        <p class="summary">Grid helps a page use rows and columns instead of only stacki
      </section>

      <section class="dashboard">
        <aside class="sidebar">
          <h2>Topics</h2>
          <a href="/layout">Layout</a>
          <a href="/spacing">Spacing</a>
          <a href="/debugging">Debugging</a>
        </aside>

        <section class="workspace">
          <section class="stats" aria-label="Lesson stats">
            <article class="stat-card">
              <span class="stat-number">12</span>
              <span class="stat-label">Lessons</span>
            </article>

            <article class="stat-card">
              <span class="stat-number">4</span>
              <span class="stat-label">Projects</span>
```

```
      </article>

      <article class="stat-card">
        <span class="stat-number">2</span>
        <span class="stat-label">Reviews</span>
      </article>
    </section>

    <section class="lesson-grid">
      <article class="lesson-card featured-card">
        <p class="card-label">Featured</p>
        <h2>Build a responsive card grid</h2>
        <p>Practice columns, gaps, and spanning so the layout feels intentional.</
      </article>

      <article class="lesson-card">
        <p class="card-label">Layout</p>
        <h2>Flexbox recap</h2>
        <p>Review parent-child layout, direct children, and one-direction alignmer
      </article>

      <article class="lesson-card">
        <p class="card-label">Grid</p>
        <h2>Track sizing</h2>
        <p>Use fixed and flexible column sizes to divide space.</p>
      </article>

      <article class="lesson-card">
        <p class="card-label">DevTools</p>
        <h2>Inspect grid lines</h2>
        <p>Use the browser overlay to see columns, rows, and gaps.</p>
      </article>
    </section>
  </section>
</section>
</main>
</body>
</html>
```

Now add the non-grid styling to **`style.css`**. This gives the page readable text, cards, borders, and spacing. The actual grid layout comes later.

css

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 1040px;
  margin: 40px auto;
}

.intro,
.sidebar,
.stat-card,
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.intro {
  margin-bottom: 24px;
}

.eyebrow,
```

```css
.card-label {
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}
```

Previous Lesson Next Lesson

```css
  line-height: 1.6;
  max-width: 640px;
}

.sidebar {
  margin-bottom: 24px;
}

.sidebar a {
  display: block;
  color: #2563eb;
  font-weight: 700;
  margin-top: 10px;
  text-decoration-thickness: 2px;
  text-underline-offset: 3px;
}

.stat-card,
.lesson-card {
  margin-bottom: 16px;
}

.stat-number {
  display: block;
  color: #2563eb;
  font-size: 32px;
  font-weight: 800;
}

.stat-label {
  color: #6b7280;
  font-weight: 700;
}

.lesson-card p {
  color: #4b5563;
  line-height: 1.6;
}
```

Open the page. It should be readable, but the layout is still a stack: intro, sidebar, stats, and lesson cards all flow downward. That is the problem Grid will solve.

Grid starts on the parent

Grid is a parent-child layout mode, like Flexbox. You put `display: grid` on a parent, and the direct children become grid items.

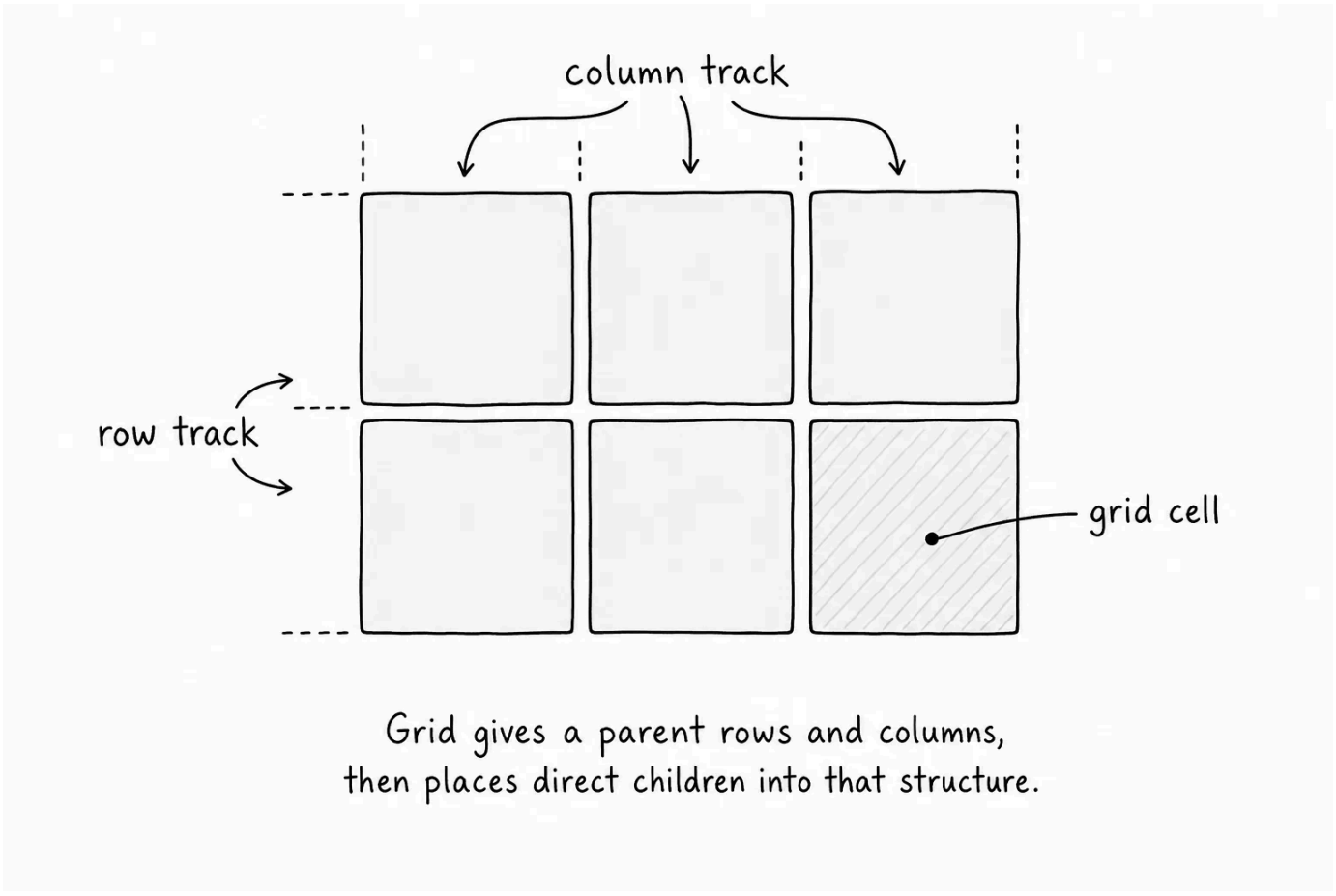
The difference is what the parent can control. A flex container controls one main direction. A grid container can define columns and rows.

css

```css
.some-parent {
```

```
  display: grid;
}
```

Grid divides a container into tracks. A column is a vertical track. A row is a horizontal track. The spaces where rows and columns cross are grid cells.



Split the dashboard into columns

The dashboard has two direct children: `.sidebar` and `.workspace`. Those two pieces should sit beside each other, so the `.dashboard` parent needs Grid.

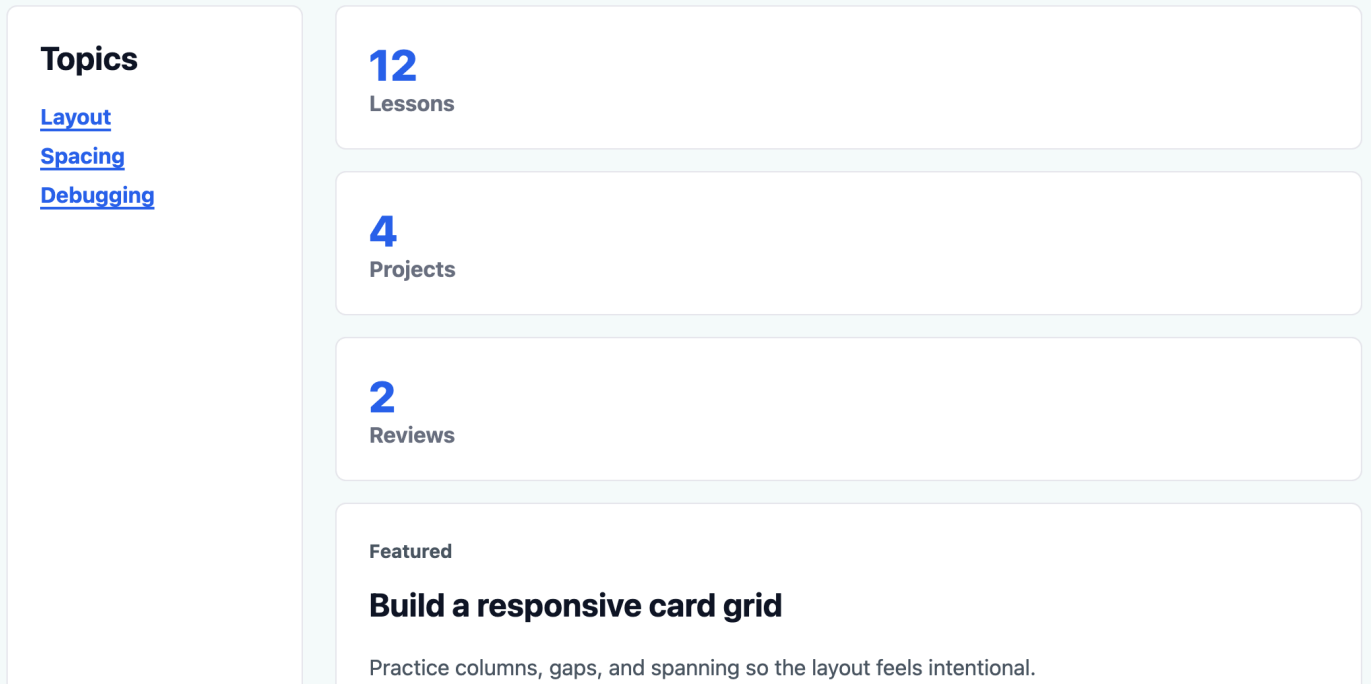
Add Grid to the dashboard parent:

```css
.dashboard {
  display: grid;
  grid-template-columns: 220px 1fr;
  gap: 24px;
}

.sidebar {
  margin-bottom: 0;
}
```

`display: grid` turns `.dashboard` into a grid container.

`grid-template-columns` defines the columns. In this case, the first column is `220px` wide for the sidebar. The second column is `1fr` for the workspace.



`fr` means a fraction of the available space. After the fixed `220px` sidebar and the gap are accounted for, `1fr` takes the remaining space.

`gap` creates space between grid tracks. You already used `gap` with Flexbox. In Grid, it creates space between rows and columns.

The `.sidebar` had `margin-bottom` in the stacked version. Once the dashboard uses a grid gap, that old margin is no longer the right spacing tool, so the rule sets it back to `0`.

This layout is intentionally desktop-shaped for now. The next lesson will teach responsive rules for smaller screens.

Build equal stat columns

The stat cards are another grid problem. There are three cards, and each should get an equal column.

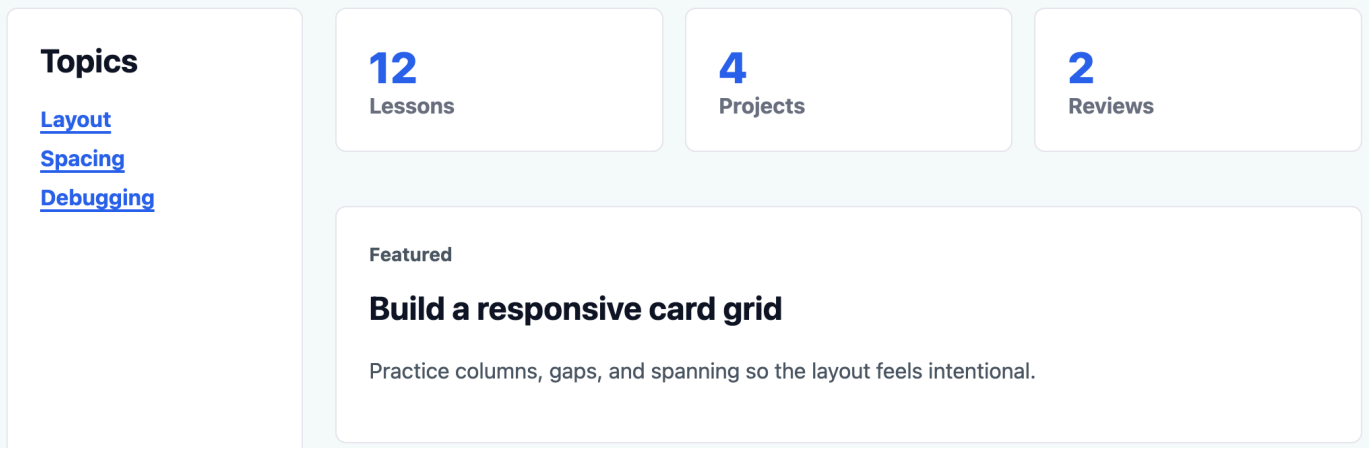
Add Grid to the stats parent:

```css
.stats {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
  margin-bottom: 24px;
}

.stat-card {
  margin-bottom: 0;
}
```

`display: grid` makes `.stats` the parent grid container. Its direct children, the three `.stat-card` elements, become grid items.

`grid-template-columns: repeat(3, 1fr);` creates three equal columns. You could write `1fr 1fr 1fr`, but `repeat(3, 1fr)` says the same thing with less repetition.



Each `1fr` column gets one share of the available space. Because all three columns use the same value, they are equal. Try playing with CSS by writing `grid-template-columns: 1fr 1fr 1fr;` and `grid-template-columns: 1fr 2fr 1fr;` and see what each produces.

The stats also use `gap` for the space between cards. The old `margin-bottom` on each `.stat-card` is removed because the grid parent now owns the spacing.

Let cards flow into rows

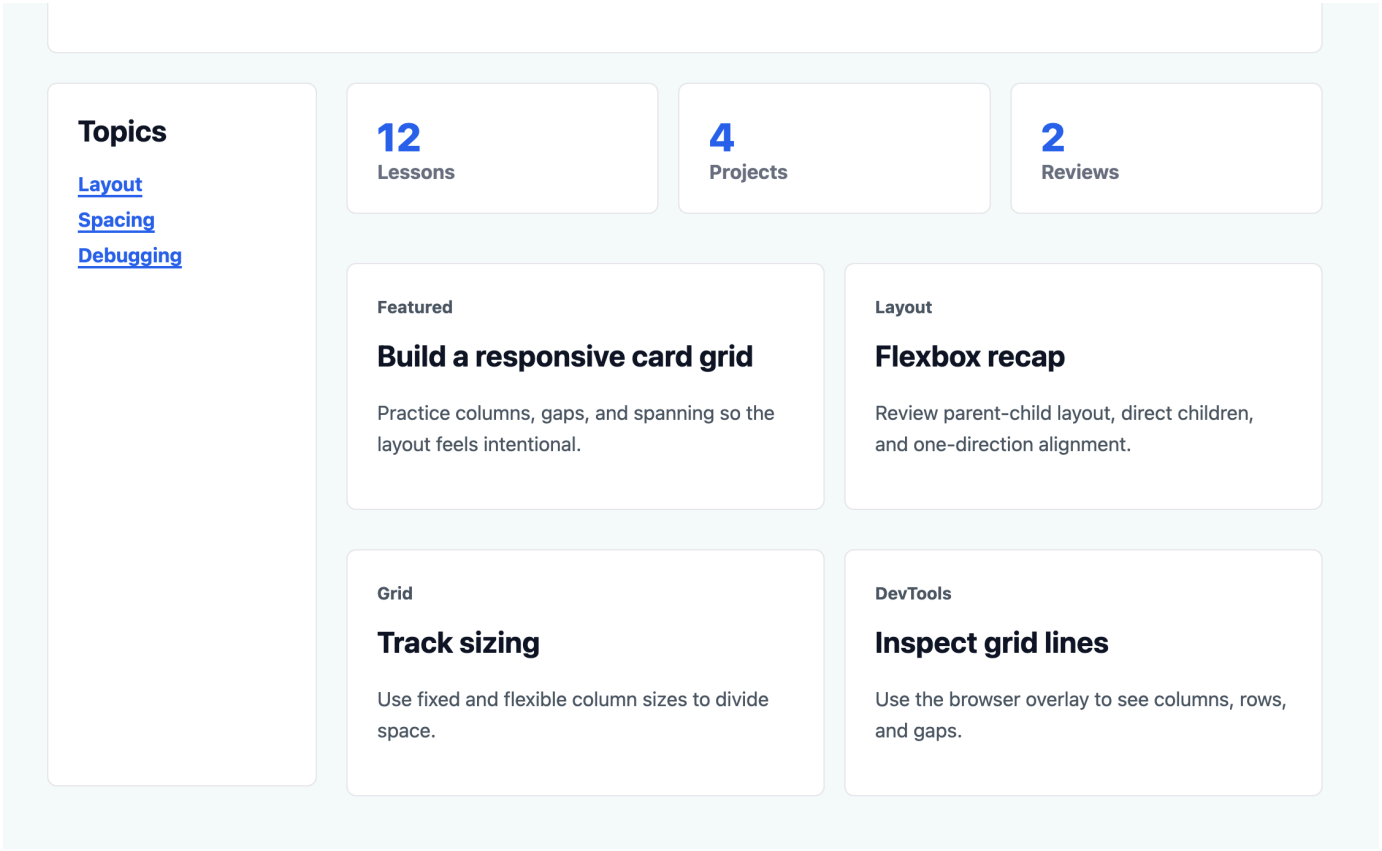
Grid can create rows automatically. If a grid has two columns and four cards, the first two cards fill the first row, and the next two cards move into the second row.

Add a two-column grid for the lesson cards:

```css
.lesson-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}

.lesson-card {
  margin-bottom: 0;
}
```

The `.lesson-grid` parent defines two equal columns. The cards are placed into those columns in HTML order.



You did not define rows here. The browser creates the rows as needed because there are more grid items than columns. This is called auto-placement: the browser places items into the next available grid cell.

> **Define the columns first** - In many beginner Grid layouts, you define the columns and let the rows appear automatically. That is enough for card grids, galleries, dashboards, and many content sections.

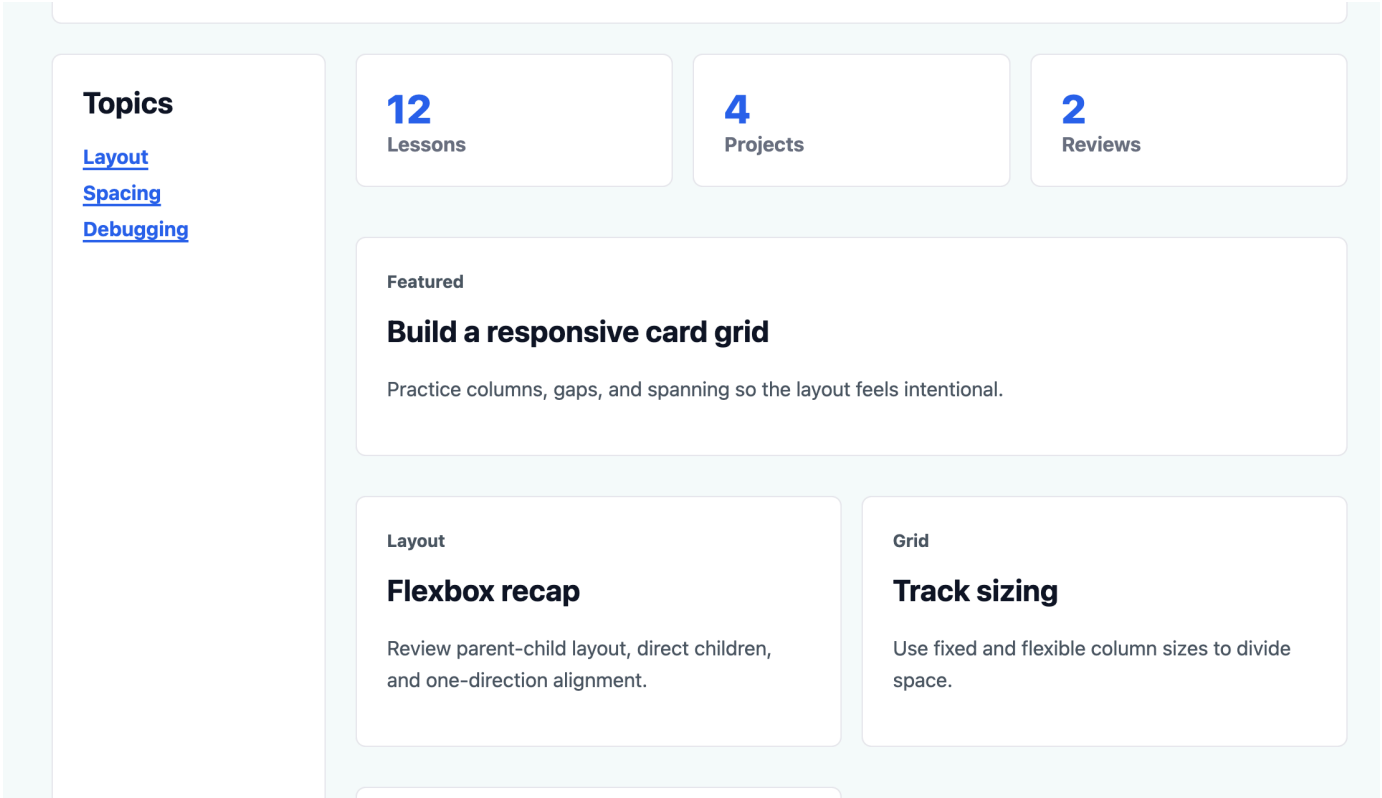
Make one card span columns

Sometimes one item should take more space than the others. In this page, the featured card can span both columns to feel more important.

Add the spanning rule:

```css
.featured-card {
  grid-column: span 2;
}
```

`grid-column` controls how a grid item uses columns. `span 2` means "take up two columns." Because `.lesson-grid` has two columns, the featured card becomes a full-width card across that grid.



The item still keeps its place in the HTML order. Grid changes where the item appears visually inside the grid, but it does not change the document meaning.

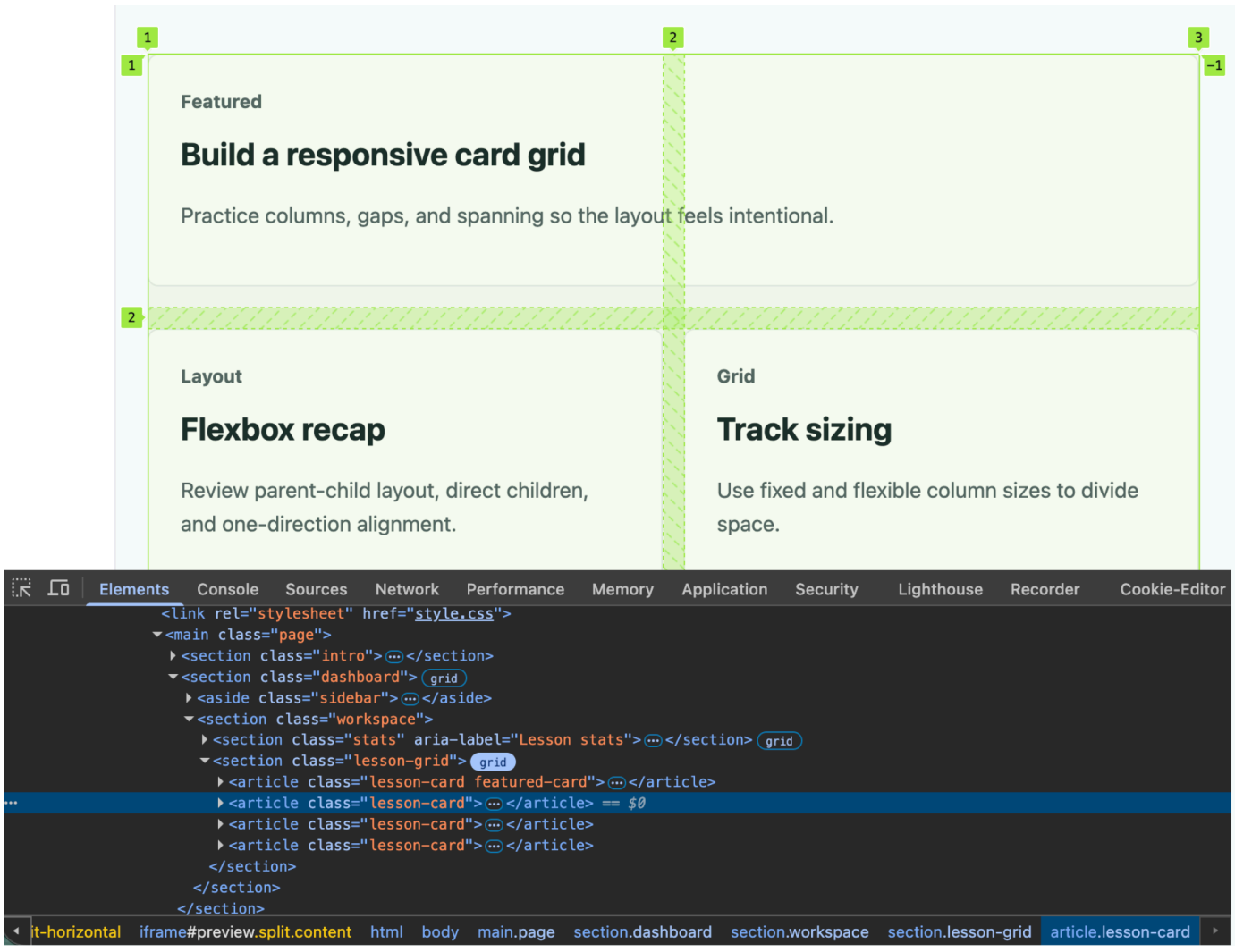
Inspect the grid in DevTools

Grid is much easier to understand when you can see the tracks.

Inspect `.lesson-grid` in DevTools. Most modern browsers show a small `grid` badge near elements with `display: grid`. Turn on the grid overlay if your browser offers it.

The overlay can show column lines, row lines, cells, and gaps. That makes it easier to confirm which parent owns the grid and where the direct children are being placed.

This is the same debugging habit from earlier lessons: inspect the actual element before guessing.



Flexbox or Grid

Flexbox and Grid are both layout tools, but they solve different problems.

Problem	Usually choose
A row of buttons	Flexbox
A card footer with text on one side and a link on the other	Flexbox
A sidebar plus main content	Grid
A card gallery with rows and columns	Grid
A dashboard with sections lined up in columns	Grid

The short version: use Flexbox when the layout mainly flows in one direction.

Use Grid when the layout needs rows and columns working together.

Put the Grid rules together

Here are the Grid rules you added:

```css
.dashboard {
  display: grid;
  grid-template-columns: 220px 1fr;
  gap: 24px;
}

.sidebar {
  margin-bottom: 0;
}
```

```css
.stats {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
  margin-bottom: 24px;
}

.stat-card {
  margin-bottom: 0;
}

.lesson-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}

.lesson-card {
  margin-bottom: 0;
}

.featured-card {
  grid-column: span 2;
}
```

The pattern is similar to Flexbox: choose the parent, turn on the layout mode, and let the direct children become layout items. The Grid-specific part is that you define columns and rows instead of only arranging along one axis.

Try it

Test Grid by changing the structure on purpose:

- Remove `display: grid` from `.dashboard`. Notice that `grid-template-columns` and `gap` no longer create the sidebar layout.
- Change `.dashboard` from `grid-template-columns: 220px 1fr;` to `grid-template-columns: 280px 1fr;`.
- Change `.stats` from `repeat(3, 1fr)` to `repeat(2, 1fr)`. Notice how the third card moves to a new row.
- Remove `gap` from `.lesson-grid`, then add it back.
- Remove `.featured-card { grid-column: span 2; }`. Notice that the featured card becomes the same width as the other cards.
- Add a fifth `.lesson-card` and reload. Watch the browser create another row automatically.
- Inspect `.lesson-grid` in DevTools and turn on the grid overlay if your browser supports it.

The goal is to see Grid as a structure tool. When a layout needs rows and columns, ask which parent owns the grid, how many columns it needs, and whether any items should span more space.

0 / 11

Knew 0 Items Learnt 0 Items Skipped 0 Items ResetProgress

Test Your Understanding

What does display: grid do?

Click to Reveal the Answer

Already Know that | Didn't Know that | Skip Question

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →